Systems programming lesson 1

COM 312: BSc Computer science/BSc IT

Topics area coverage.

- **Brief history of Linux**
- **Design principles of Linux**
- **Overview of what is systems programming**
- **Components of systems programming**
- **Theory and practical of systems call using C programing language.**

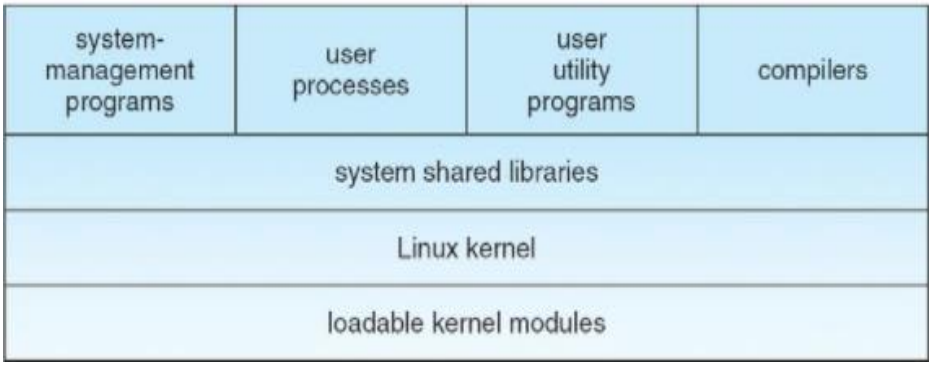
Brief history of Linux

- **Linux is a modern, free operating systems based on UNIX standards.**
- **First developed as a small but self -contained kernel in 1991 by Linus Torvalds with major design goal of UNIX compatibility.**

Design principles of Linux

- **Main design goals are speed, efficiency, and standardization**
- **Linux is a multi-user, multitasking systems with a full set of UNIX – compatible tools.**
- **Linux is designed to be compliant with the relevant POSIX documents (*portable operating system interface*) at least two Linux distributors have achieved official POSIX certification.**
- **The Linux programming interface adheres to the SVR4 UNIX semantics, rather than to BSD behavior**

COMPONENTS OF LINUX SYSTEMS



Linux system management - refers to the ability to manage Linux-based systems such as desktops, laptops, and servers

User processes- A Linux process is nothing but running instance of a program. For example, when you start Firefox to browse Internet, you can create a new process. In Linux, each process is **given a unique number** called as a process identification (PID)

User utility programs- Utility programs that **provide the user most of the functionalities of an operating systems**.

Compilers - computer software that translates (compiles) source code written in a high-level language (e.g., C++) into a set of machine-language instructions that can be understood by a digital computer's CPU.

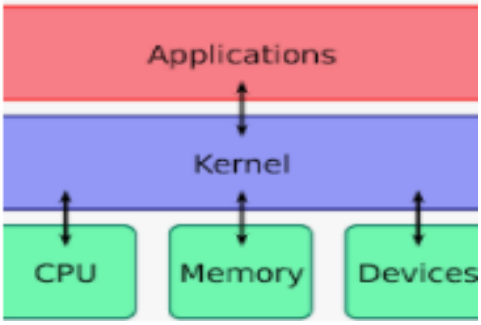
System shared libraries- Shared Libraries are **the libraries that can be linked to any program at run-time**. They provide a means to use code that can be loaded anywhere in the memory. Once loaded, the shared library code can be used by any number of programs

Components of A Linux System

- Linux system is composed of **three main bodies of code**, in the line with the most traditional UNIX implementations.
  i. Kernel
  ii. System libraries
  iii. System utilities

1. Kernel

The kernel is responsible for maintaining all the important **abstractions** of the operating systems, including such things as virtual memory and processes.



Kernel is an **interface** between application and actual data processing at the hardware level.

The kernel connects the system hardware to the application software.

- **The more internal processes are provided by the kernel**
- **Whenever a system starts, the kernel is the first program that is loaded after the bootloader because the kernel has to handle the rest of the thing of the system for the operating system.**

- **The kernel remains in the memory until the operating system is shut down.**

Typically, kernel is responsible for low –level task (or you can say the basic functions of kernel are):-

- **Access Computer resource:** A Kernel can access various computer resources like the CPU, I/O devices and other resources. It acts as a bridge between the user and the resources of the system.
- **Resource Management:** It is the duty of a Kernel to share the resources between various process in such a way that there is uniform access to the resources by every process.
- **Memory Management:** Every process needs some memory space. So, memory must be allocated and deallocated for its execution. All these memory management is done by a Kernel.
- **Device Management:** The peripheral devices connected in the system are used by the processes. So, the allocation of these devices is managed by the Kernel.

Kernel Mode and User Mode

There are certain instructions that need to be executed by **Kernel only**. So, the CPU executes these instructions in the Kernel Mode only. For example, memory management should be done in Kernel-Mode only. While in the User Mode, the CPU executes the processes that are given by the user in the User Space.

2.System Library

System libraries are special functions or programs using which application programs or system utilities accesses Kernel's features. These libraries implement most of the functionalities of the operating system and do not requires kernel module's code access rights.

3.System Utility

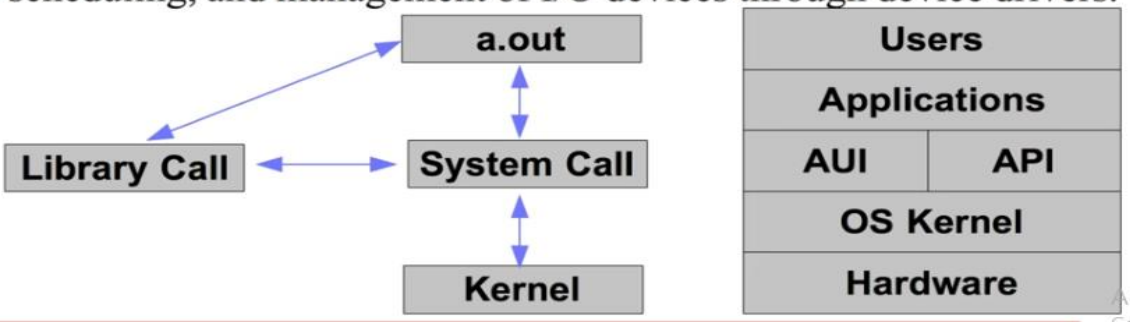
System Utility programs are responsible to do specialized, individual level tasks.

Introduction to systems programming and essential concepts

In computer there are two types of software

1. **Systems software**
2. **Application software**

- ***Application programming*** aims to produce software which provides services to the user, e.g., word processors, browsers, multi-media applications, etc.
- ***System Programming*** aims to produce software which provides services to computer hardware, e.g., disk defragmenter. The next step is writing kernel code to manage main memory, disk space management, cpu scheduling, and management of I/O devices through device drivers.



a.out – executable file access kernel services while at library call which farther calls system call to access kernel.

System Programmer Perspective

- A system programmer write programs that may have to acquire data
  - from a file(s) (residing on a local or remote disc) that may have been opened by some other user(s),
  - from other programs running on the same or different machine,
  - from the operating system itself.
- After processing, the programs may have to write results to a shared resource, which other processes are also writing, or results may need to be delivered to another process asynchronously, i.e., not when the process asked for it, but at some later unpredictable time. **The challenge is to master these concepts.**

Operating System Services

Some important tasks a kernel performs are:

- File Management
- Process management
- Memory management
- Information management
- Signal handling
- Synchronization
- Communication (IPC)
- Device management

Two methods by which a program can make requests for services from the Kernel:

- By making a system call (entry point built directly into the kernel)
- By calling a library routine that makes use of this system call

Activate Win

What is systems programming?

- **system programming- is the art of writing system software.**

System software lives at a low level, interfacing directly with the kernel and core system libraries.

Benefits of learning system programming

- **you get a better understanding of how the operating system works. this can ultimately help you in developing efficient programs.**
- **Cloud computing, high performance computing, distributed systems are some of the areas where system programming is heavily used.**
-

Systems call

system programming starts with system calls.

What is systems call?

- **A system call is a controlled entry point into the kernel, allowing a process to request that the kernel perform some action on the process's behalf. why?**
  It is not possible to directly link user-space applications with kernel space. For reasons of security and reliability, user-space applications must not be allowed to directly execute kernel code or manipulate kernel data

- **The kernel makes a range of services accessible to programs via the system call application programming interface (API). These services include, for example, creating** a new process, performing I/O, and creating a pipe for interprocess communication.
-

Why do we need systems calls in OS?

The two main reasons why such instructions are not available directly for the process.

- **(i) protection**

  The OS must do many checks to evaluate the right of the process to request such an instruction.

  (ii) update

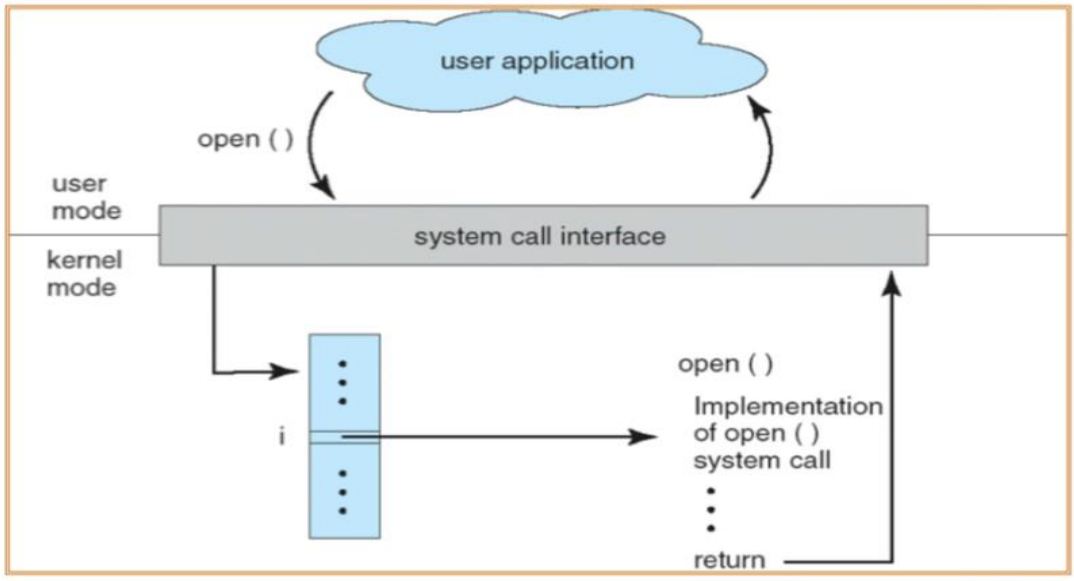
  The OS needs to update its data structures when the instruction is performed. If we take folk system call, the needs to update many data structures related to processes to take into account the new process, add it in the running queue, etc.

- **Reading and writing from files demand system calls.**
- **If a file a system wants to create or delete files, systems calls are required.**
- **System calls are used for creation and management of new processes.**
- **Network connections need systems for sending and receiving packets.**
- **Access to hardware devices like scanner, printer, need a system call.**

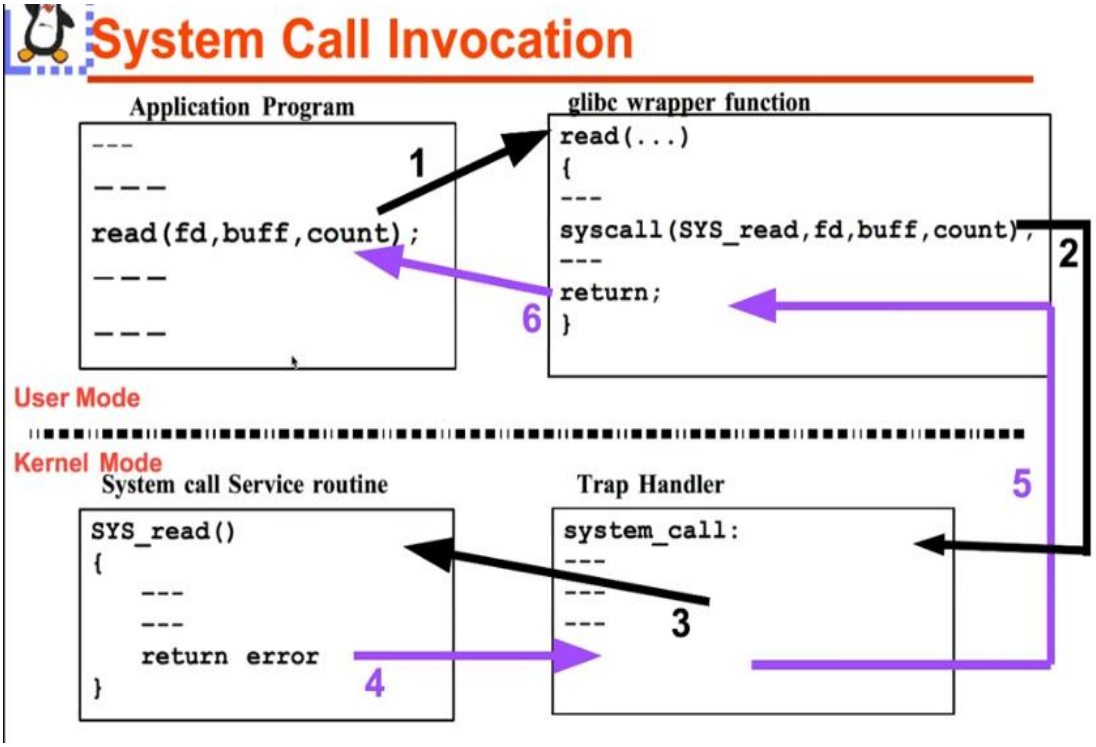
Before going into the details of how a system call works, we note some general points:

- A system calls changes the processor state from user mode to kernel mode, so that the CPU can access protected kernel memory.
- The set of system calls is fixed. Each system call is identified by a unique number. (This numbering scheme is not normally visible to programs, which identify system calls by name.)
- Each system call may have a set of arguments that specify information to be transferred from user space (i.e., the process's virtual address space) to kernel space and vice versa.

System Call Invocation



System Call Invocation



It is possible to invoke system calls directly by system call but since most program rely on glibe functions for system call.

- **From a programming point of view, invoking a system call looks much like calling a C function.**
  1. **The application program makes a system call by invoking a wrapper function in the C library**

2. **The wrapper function must make all of the system call arguments available to the system call trap-handling routine (described shortly). These arguments are passed to the wrapper via the stack, but the kernel expects them in specific registers. The wrapper function copies the arguments to these registers.**
3. **Since all system calls enter the kernel in the same way, the kernel needs some method of identifying the system call. To permit this, the wrapper function copies the system call number into a specific CPU register (%eax).**
4. **The wrapper function executes a trap machine instruction (int 0x80), which causes the processor to switch from user mode to kernel mode and execute code pointed to by location 0x80 (128 decimal) of the system's trap vector.**
5. **In response to the trap to location 0x80, the kernel invokes its system_call() routine (located in the assembler file arch/i386/entry.S) to handle the trap. This handler: a) Saves register values onto the kernel stack (Section 6.5). b) Checks the validity of the system call number. c) Invokes the appropriate system call service routine, which is found by using the system call number to index a table of all system call service routines (the kernel variable sys_call_table). If the system call service routine has any arguments, it first checks their validity; for example, it checks that addresses point to valid locations in user memory. Then the service System Programming Concepts routine performs the required task, which may involve modifying values at addresses specified in the given arguments and transferring data between user memory and kernel memory (e.g., in I/O operations). Finally, the service routine returns a result status to the system_call() routine. d) Restores register values from the kernel stack and places the system call return value on the stack. e) Returns to the wrapper function, simultaneously returning the processor to user mode.**
6. **If the return value of the system call service routine indicated an error, the wrapper function sets the global variable errno (see Section 3.4) using this value. The wrapper function then returns to the caller, providing an integer return value indicating the success or failure of the system call.**

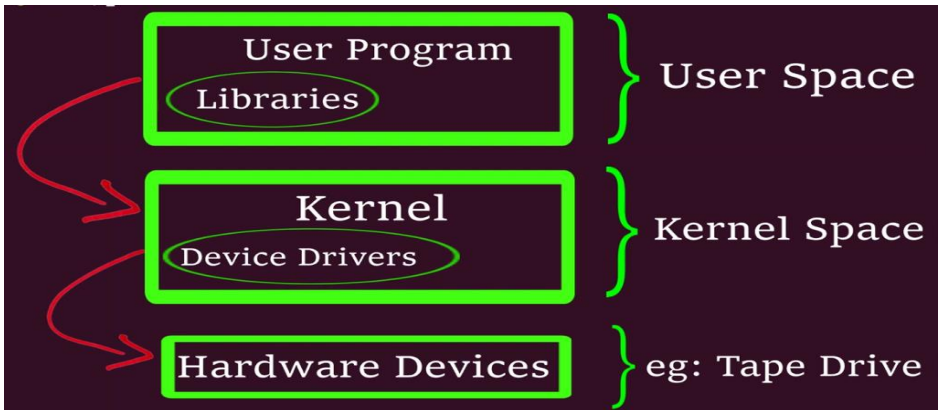
System calls (Read, write, open, close) *low level functions* program code illustration using Linux.

Open- opens a file /device

Read- reads from an open file

Write- write to a file/device

Close- close a file/ device



how to Access Tape drive from user space mode we need a system call via kernel.

Systems call program.

```c
#include<unistd.h>
#include<stdio.h>
#include<fcntl.h>

int main()
{
int fd;
char buffer[80];
char msg[50] = "hello ifocus institute";
fd = open ("check.txt", O_RDWR);
printf("fd = %d",fd);
if (fd != -1)
{
printf("\n check.txt opened with read write access\n");
write(fd, msg , sizeof(msg));
lseek(fd,0,SEEK_SET);
read(fd, buffer, sizeof(msg));
printf("\n %s was written to my file\n", buffer);
close (fd);
}

return 0;
}
```

Systems call functions highlighted.

Code explained

1. **#include<unistd.h> - name of the header file that provides access to the POSIX (***portable operating system interface is a family of standards specified by*

IEEE for maintaining compatibility between operating systems) **operating API.**
i.e close function is declared in unistd.h

The file contains the system call number that is passed to the kernel through the register(EAX) when a system call is invoked.

2. **#include<fcntl.h>- file control options**
   fcntl.h *function can be used to change the properties of a file that is already open.*

i.e. file descriptor flags.

3. line line 7: **int fd (file descriptor)- integer that uniquely identifies an open file of the process.**

Standard file descriptors – when any process starts FD 0,1,2 open automatically.

Read from stdin => read from fd 0: whenever we write any character from keyboard it read keyboard, it read from stdin through fd 0.

Write to stdout => write to fd1: whenever we see any output to the video screen, its from the file named /dev/tty and written to standard out in screen **through fd1.**

Write to stderr => write to fd2: we see any error to the video screen, it is also from that file write to stderr in the screen through fd2.

3. line 10 : fd =open(" check.txt",0_RDWR);

```
NAME
       open, openat, creat - open and possibly create a file

SYNOPSIS
       #include <sys/types.h>
       #include <sys/stat.h>
       #include <fcntl.h>

       int open(const char *pathname, int flags);
       int open(const char *pathname, int flags, mode_t mode);
```

Open system call function- has two parameters * pathname, int flags.

```
The argument flags must include one of the following access modes: O_RDONLY, O_WRONLY, or O_RDWR.  These
request opening the file read-only, write-only, or read/write, respectively.
```

```
 7 int fd;
 8 char buffer[80];
 9 char msg[50] = "hello ifocus institute";
10 fd = open ("check.txt", O_RDWR);
11 printf("fd = %d",fd);
12 if (fd != -1)
```

Line 12 : if(fd!= -1)

If the systems call fails, the it returns an error…but in our program its successful no error.

Line 15 : `write(fd, msg , sizeof(msg));`

```
SYNOPSIS
       #include <unistd.h>

       ssize_t write(int fd, const void *buf, size_t count);
```

Write function call has three parameters

Fd -

Buffer – where the read data is to be stored and the number of bytes to be read from the file. (array of bytes unlike string) buffers are low level and can be can refer to any data type.

Count – number of bytes and store them into the buffer.

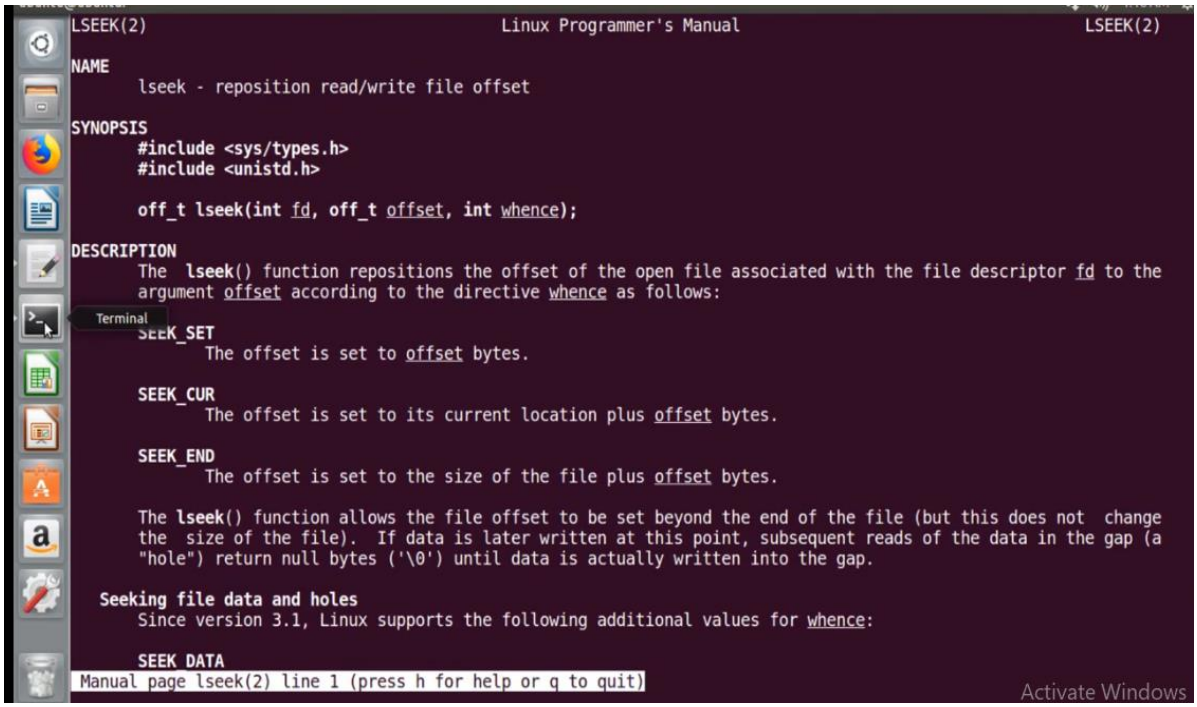
Line 16 : `lseek(fd,0,SEEK_SET);`

Lseek- used to change location of Read/Write pointer of a file descriptor

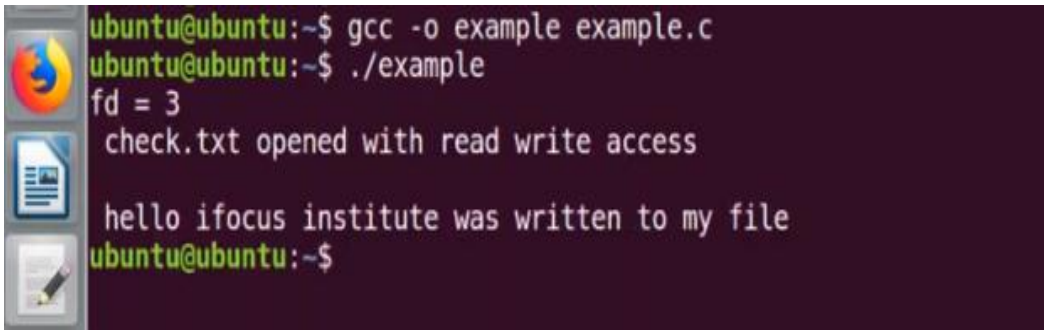


Just click on terminal type man followed by [linux command name]

Man means user linux manual **i.e man lseek.**



Output of the program

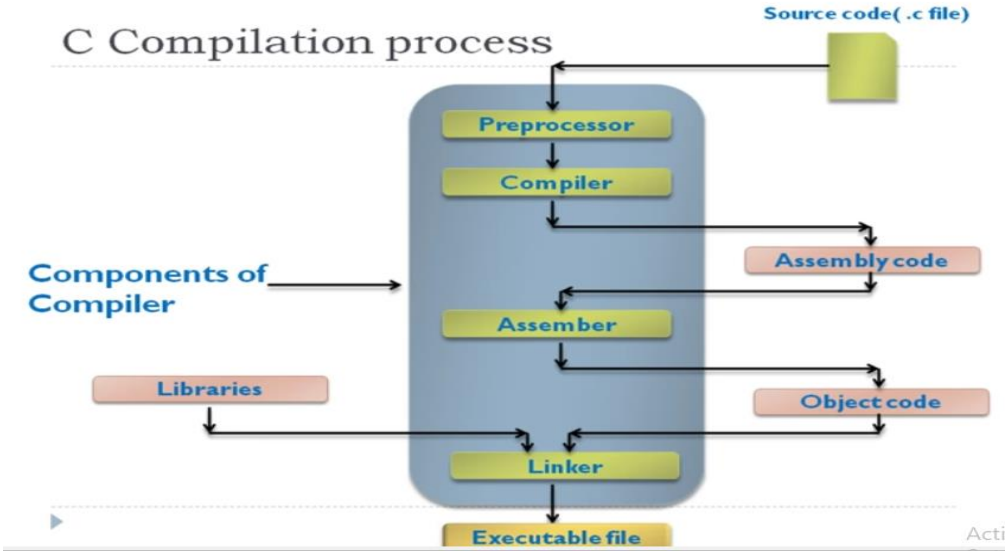


Compiling C files with Gcc compiler using Linux

Compilation process

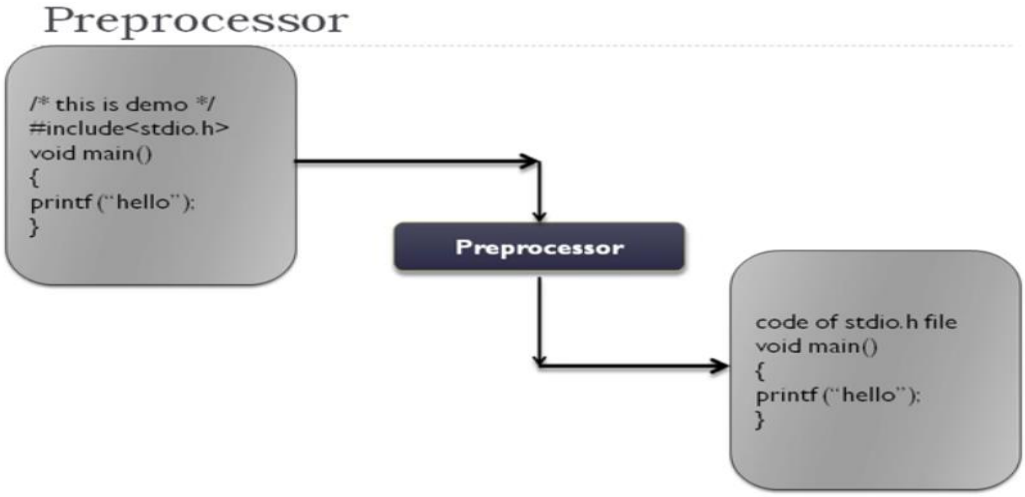
Four Steps of Compilation:

i. preprocessing,
ii. compiling,
iii. assembly,
iv. linking.

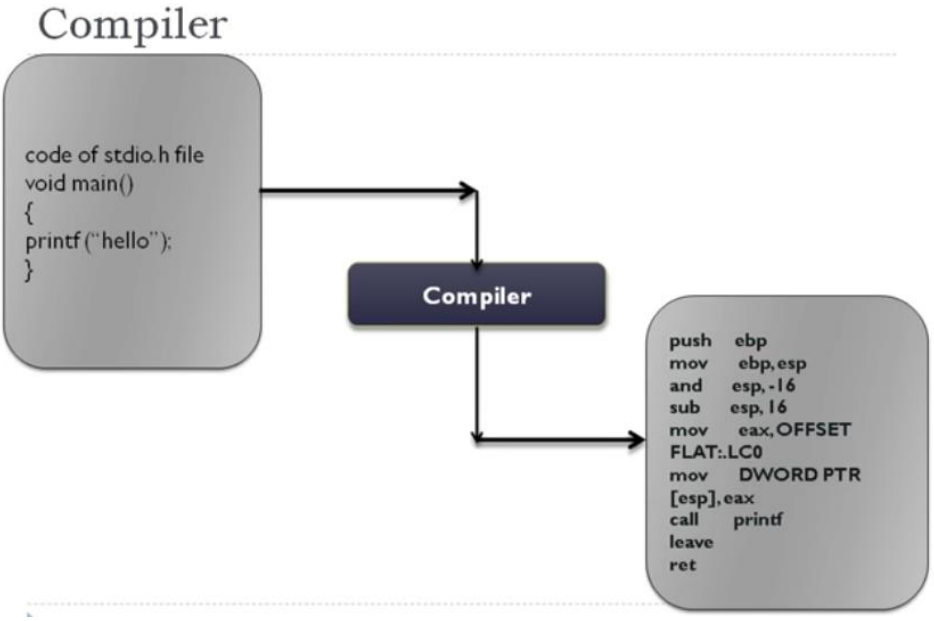


1. **Preprocessing**. The preprocessor obeys commands that begin with # (known as directives) by:

- removing comments
- expanding macros
- expanding included files



2. **compiling -** It takes the output of the preprocessor and generates assembly language, an intermediate human readable language, specific to the target procer

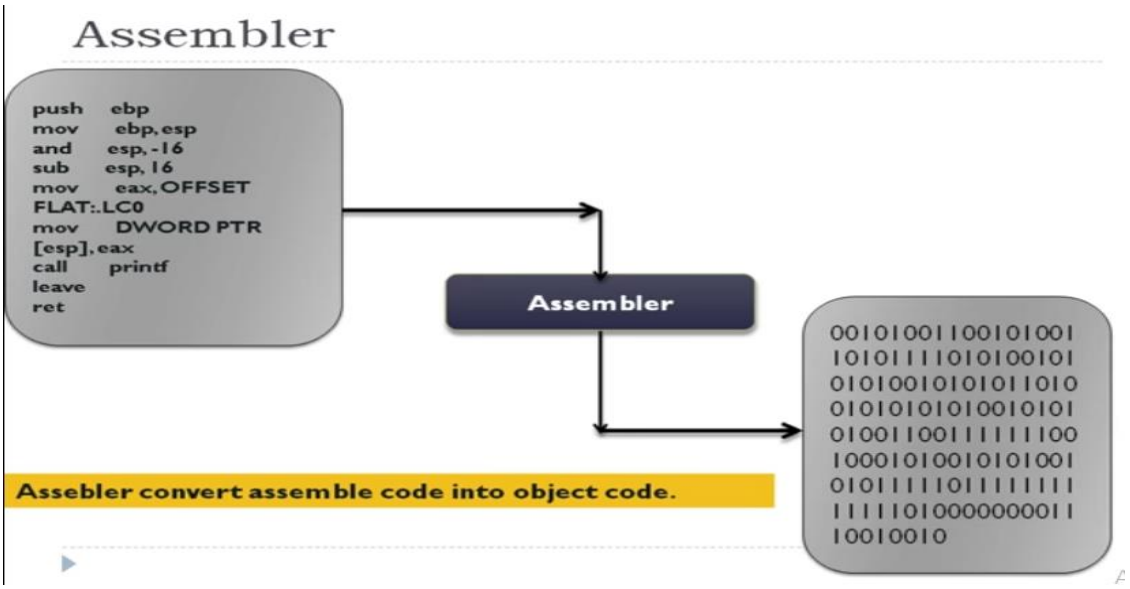


Why Assembly Language?

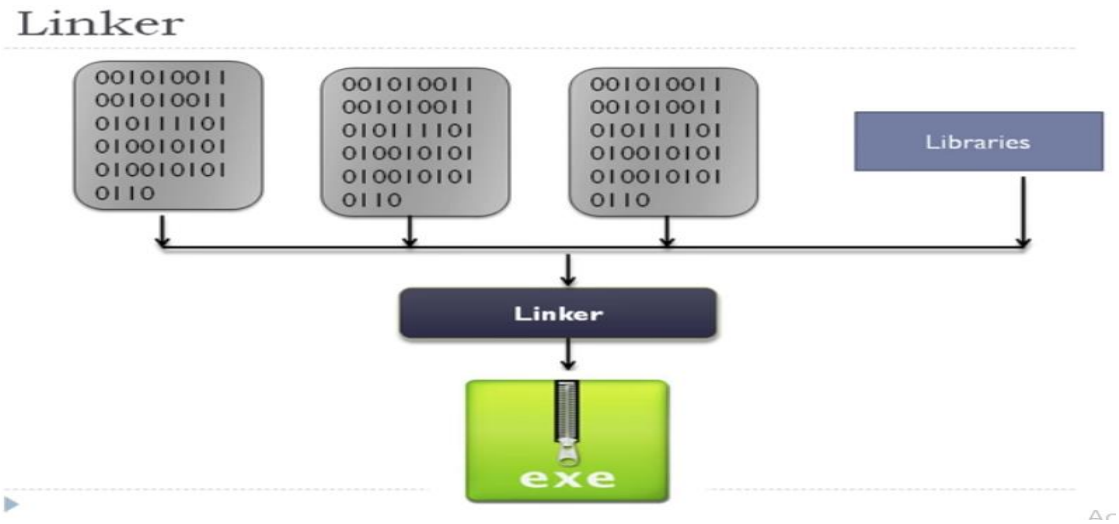
- By learning assembly language, the programmer can write the code to access registers and retrieve the memory address of pointers and values. It mainly helps in speed optimization that increases efficiency and performance.

- Assembly language learning helps in understanding the processor and memory functions. If the programmer is writing any program that needs to be a compiler, that means the programmer should have a complete understanding of the processor. Assembly language helps in understanding the work of processors and memory. It is cryptic and symbolic language.

3. **Assembler**



4. **Linker**



How to Install Kali Linux 2020.1b in VirtualBox on Windows 10
https://www.youtube.com/watch?v=V_PayI5FlgQ&t=5s

lesson 2: we will start with our system programming practical's using **LINUX OS and C programming language.**

using kali linux version because of Network security topic.

NB: any other LINUX version is okay

i. **Redhat**
ii. **Ubuntu**
iii. **Fedora etc ..but for class demos Kali Linux**